REAL-TIME OPERATING SYSTEMS

Detailed Study Guide for External Examination

Course Outcomes **CO1** & **CO2** — Complete Question Bank with In-Depth Answers

This guide gives full, exam-length answers for every question: precise definitions, detailed explanations, labelled diagrams, formulas, step-by-step worked solutions for all numerical problems, comparison tables, advantages & disadvantages, and exam-writing tips. Written so a student can reproduce a complete, high-scoring answer in the examination.

Part	Section	No. of Questions
CO1	Short Answer Questions	9
CO1	Long Answer Questions	18
CO2	Short Answer Questions	9
CO2	Long Answer Questions	17
	TOTAL QUESTIONS	**53**

How to use this guide: read the **Definition** box first, then the explanation and diagram, and practise every numerical trace by hand at least once. In the exam, always state the definition, draw the diagram/table, show the working, and box the final answer.

Contents

What this guide covers, in order

Section	Contents
How to Write High-Scoring Answers	Exam strategy for theory and numericals
CO1 – Short Answers (9)	OS fundamentals, RTOS basics, OS types
Concept Primer: CPU Scheduling	Scheduling basics + master comparison table
Concept Primer: Memory Management	Allocation, fits, fragmentation
CO1 – Long Answers (18)	Scheduling & memory problems (worked)
Concept Primer: Virtual Memory	Paging, TLB, replacement, thrashing
CO2 – Short Answers (9)	TLB, address space, tasks, APIs
CO2 – Long Answers (17)	Page replacement + VM/IPC/segmentation
Appendix A	Quick comparison tables
Appendix B	Formula & fact sheet
Appendix C	Glossary of key terms
Appendix D	Likely exam & viva questions
Appendix E	Extra solved practice problems
Appendix F	Common mistakes to avoid

Course Outcome coverage: CO1 focuses on operating-system fundamentals, CPU scheduling and basic memory management; **CO2** focuses on virtual memory, paging, segmentation, real-time tasks and inter-process communication. Every question from the bank is answered in full, in the order listed above.

How to Write High-Scoring Answers

A quick strategy page before the answers

For theory (short & long answers)

1. **Start with a one-line definition** of the key term – examiners award the first mark for this.
2. **Draw the diagram** wherever one exists (process states, paging, TLB, segmentation). A neat labelled diagram earns easy marks and saves words.
3. **Use headings / points** rather than one long paragraph – it is easier to mark and shows structure.
4. **Give one real example** (e.g. an RTOS name, a real device) to show application.
5. **End with advantages/disadvantages or a comparison table** where relevant.

For numerical problems (scheduling, paging, memory)

1. **Rewrite the given data** (process/arrival/burst table or the reference string) so the examiner sees your input.
2. **State the rule/formula** you are applying (e.g. 'FIFO evicts the oldest page').
3. **Draw the Gantt chart or the frame table** showing every step – most marks are for the working, not the final number.
4. **Write the formulas:** Turnaround = Completion – Arrival; Waiting = Turnaround – Burst; Hit ratio = hits / total references.
5. **Box the final answer** (average waiting time, total page faults, etc.).

> **EXAM TIP**
> A common mistake is writing only the final number. Always show the table/Gantt chart and the formula substitution – a correct answer with no working often loses more than half the marks.

CO1 — Short Answer Questions

Operating-system fundamentals, RTOS basics and OS classification

Q1. What are the basic functionalities of an Operating System and their role in efficient system operation?

Concept: *OS as resource manager & extended machine* | ***Key terms:*** *kernel, abstraction, resource management*

> **DEFINITION**
> An **Operating System (OS)** is the system software that acts as an interface between the user/application programs and the computer hardware. It controls and coordinates the use of hardware among the various application programs for the different users.

An OS plays **two fundamental roles**. As an **extended (virtual) machine** it hides the messy details of the hardware behind clean, easy-to-use abstractions (files instead of disk sectors, processes instead of raw CPU time). As a **resource manager** it allocates the processor, memory, and I/O devices fairly and efficiently among competing programs.

Basic functionalities

- **Process management** – creation, scheduling, synchronisation, communication and termination of processes; decides which process uses the CPU and when.
- **Memory management** – keeps track of every byte of memory, allocates space to processes, and provides virtual memory so programs larger than RAM can run.
- **File management** – organises data into files and directories, and controls creation, deletion and access permissions.
- **Device (I/O) management** – manages device drivers, buffering, caching and spooling so that slow devices do not stall the CPU.
- **Security and protection** – authenticates users and isolates processes so one cannot corrupt another.
- **Command interpretation / user interface** – provides a shell or GUI through which users issue commands.

Role in efficient system operation

By **scheduling the CPU**, **overlapping I/O with computation**, and **reclaiming free memory**, the OS keeps expensive hardware busy and allows many programs to share a single machine without interfering with one another. Efficient resource utilisation, higher throughput and protection are the direct results.

> **EXAM TIP**
> Remember the two roles ('extended machine' + 'resource manager') and list at least five functionalities with one line each. Naming the roles first usually secures the opening marks.

Q2. What are the major components of an RTOS architecture and their role in deterministic task execution?

Concept: *RTOS building blocks* | ***Key terms:*** *scheduler, dispatcher, IPC, ISR, timing services*

> DEFINITION
> A **Real-Time Operating System (RTOS)** is an OS designed to serve real-time applications that process data as it comes in, typically within strict **timing constraints (deadlines)**. Its architecture is organised so that the worst-case behaviour of every component is **bounded and predictable**.

Major components

- **Real-time scheduler** – usually priority-based and pre-emptive (e.g. Rate-Monotonic or Earliest-Deadline-First); selects the ready task that must run so that deadlines are met.
- **Dispatcher** – carries out the context switch to the chosen task with minimal, bounded latency.
- **Task (thread) management** – maintains Task Control Blocks and the state of every task (ready, running, blocked, suspended).
- **Inter-task communication & synchronisation** – message queues, semaphores and mutexes, often with **priority inheritance** to prevent priority inversion.
- **Interrupt Service Routines (ISRs)** – provide fast, bounded response to external hardware events.
- **Timer / clock services** – supply the system tick and time management on which all timing guarantees depend.
- **Memory management** – often simple and static to keep allocation time predictable.

Role in determinism: because each component has a **bounded worst-case execution time**, the system's overall response can be *guaranteed* in advance rather than merely being fast on average. This predictability is what separates an RTOS from a general-purpose OS.

Examples of RTOS: FreeRTOS, VxWorks, QNX, RTLinux.

> EXAM TIP
> List the scheduler, dispatcher, IPC, ISR and timer as the 'big five' components and stress the word **bounded / predictable** for the determinism part.

Q3. What is the role of the Operating System kernel in managing processes, memory and system resources?

Concept: *The kernel* | ***Key terms:*** *privileged mode, system calls, core services*

> DEFINITION
> The **kernel** is the central core of the operating system that is always resident in main memory and runs in **privileged (supervisor) mode** with direct control of the hardware. It provides the most basic services on which everything else is built.

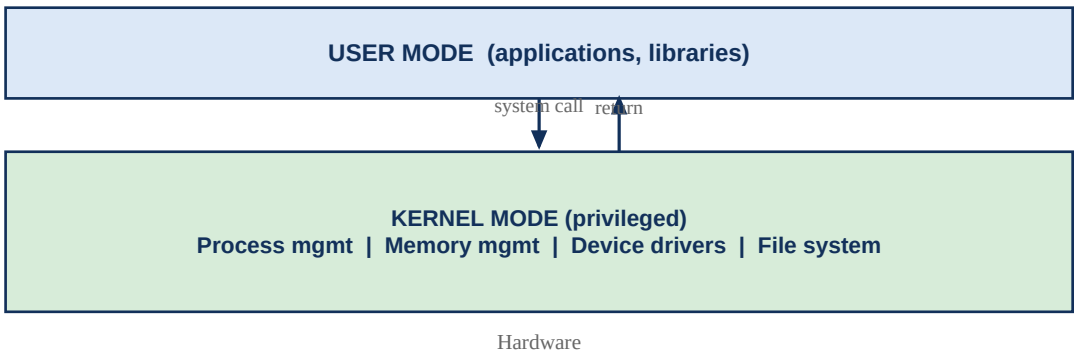


Fig: User mode vs kernel mode. Programs request privileged services through system calls.

Roles of the kernel

- **Process management** – creates and terminates processes, schedules the CPU and performs context switching between processes.
- **Memory management** – allocates and frees memory, maintains page tables and enforces address-space protection between processes.
- **Resource & device management** – arbitrates access to I/O devices, disks and files through device drivers.
- **System-call interface** – offers a controlled 'gateway' through which user programs request privileged operations safely.
- **Interrupt handling** – responds to hardware and software interrupts.

By concentrating all privileged operations inside one protected component, the kernel guarantees **controlled, safe and efficient sharing** of every system resource. Kernels may be **monolithic** (all services in one large kernel, e.g. Linux) or **micro-kernels** (minimal kernel with services as user processes, common in RTOS for reliability).

Q4. List two common application areas of RTOS and mention why timing is important in those applications.

Concept: *RTOS applications* | ***Key terms:*** *hard real-time, deadlines, safety-critical*

An RTOS is used wherever the **correctness of the result depends on the time** at which it is produced. Two common application areas:

1. Automotive & avionics control systems

Examples: anti-lock braking systems (ABS), engine control units, and aircraft flight-control systems. Sensor readings must be processed and actuators driven within a few **milliseconds**. A late response – even by a fraction of a second – can cause loss of control, so these are **hard real-time** systems.

2. Medical devices

Examples: pacemakers, infusion pumps and patient monitors. Electrical stimulation or drug dosing must occur at **precise instants**; incorrect timing can directly endanger the patient's life.

Why timing is important

- In such systems, a correct answer delivered **too late is equivalent to a wrong answer**.
- Missed deadlines may cause **catastrophic failure** (hard real-time) or serious quality loss (soft real-time).

- Therefore the OS must guarantee **bounded, predictable response times**, not just high average speed.

Other application areas: industrial robotics, telecommunications, defence radar, and process control plants.

Q5. Compare general-purpose and real-time operating systems with respect to response-time requirements.

Concept: *GPOS vs RTOS* | ***Key terms:*** *throughput vs determinism*

A **General-Purpose OS (GPOS)** is optimised for good *average* performance and user experience, whereas an **RTOS** is optimised for meeting *deadlines* with predictable timing.

Aspect	General-Purpose OS (GPOS)	Real-Time OS (RTOS)
Primary goal	High throughput, fairness, good user experience	Meeting deadlines and predictability
Response time	Best-effort, variable (not guaranteed)	Bounded and guaranteed
Scheduling	Fairness-oriented (round robin, CFS)	Priority / pre-emptive (RMS, EDF)
Determinism	Low	High (deterministic)
Kernel	Usually monolithic, larger latency	Small, low-latency, often micro-kernel
Time management	Coarse	Fine, tick-based
Examples	Windows, Linux, macOS	FreeRTOS, VxWorks, QNX

In summary: a GPOS gives the best *average* response, while an RTOS guarantees the *worst-case* response so that every task provably meets its deadline.

Q6. Differences between batch, time-sharing and real-time operating systems based on execution characteristics.

Concept: *OS types by execution* | ***Key terms:*** *batch, interactive, deadline-driven*

Feature	Batch OS	Time-Sharing OS	Real-Time OS
User interaction	None; jobs are queued and run in batches	Highly interactive; many users at once	Event / deadline driven
Main goal	Maximum throughput	Fast response to many users	Meet strict deadlines
CPU sharing	One job runs to completion	Time slices (round robin)	Priority-based pre-emption
Response time	Not important	Should be small (fair)	Bounded & guaranteed
Examples	Payroll, bank statements	UNIX, multi-user Linux	Flight control, robotics

Batch OS processes grouped jobs with no interaction; **time-sharing OS** rapidly switches among users to give each the illusion of a dedicated machine; **real-time OS** reacts to events within guaranteed deadlines.

Q7. What is an RTOS and state the importance of timing constraints in an RTOS.

Concept: *RTOS definition* | ***Key terms:*** *deadline, hard/soft/firm real-time*

> **DEFINITION**
>
> A **Real-Time Operating System (RTOS)** is an operating system that guarantees to complete tasks within specified **time constraints (deadlines)**, so that logical correctness depends not only on *what* result is produced but also on *when* it is produced.

Types of real-time systems

- **Hard real-time:** missing a deadline is a total, often catastrophic failure (e.g. airbag deployment, flight control).
- **Firm real-time:** a missed deadline makes the result useless, but no catastrophe occurs (e.g. a frame dropped in a live analysis).
- **Soft real-time:** deadlines should be met, and occasional misses only degrade quality (e.g. video/audio streaming).

Importance of timing constraints

- Timing **determinism** is the defining property that separates an RTOS from an ordinary OS.
- The **scheduler, ISRs and IPC** are all designed to have bounded worst-case latency.
- In safety-critical systems, a late result can cause **loss of life or equipment**.
- A **schedulability analysis** is performed in advance to prove all deadlines can be met.

Q8. Explain the major functions performed by an Operating System in managing system resources.

Concept: *Resource-management view* | ***Key terms:*** *CPU, memory, I/O, files, protection*

Viewed as a **resource manager**, the OS keeps track of resources, decides who gets what, allocates them, and reclaims them. Its major resource-management functions are:

1. **Processor (CPU) management** – scheduling processes onto the CPU, deciding order and duration, and performing context switches to maximise utilisation.
2. **Memory management** – tracking which parts of memory are used, allocating/de-allocating space, and providing protection and virtual memory.
3. **Device management** – controlling I/O devices via drivers, using buffering, caching and spooling.
4. **File management** – creating, storing, retrieving and protecting files, and managing directory structures.
5. **Security & protection** – authenticating users and isolating processes from one another.
6. **Accounting & coordination** – recording resource usage and resolving conflicts among competing requests.

Q9. Classify the major types of Operating Systems and state one distinguishing characteristic of each.

Concept: *OS taxonomy* | ***Key terms:*** *batch, multiprogramming, time-sharing, distributed, RTOS*

Type of OS	Distinguishing characteristic

Batch OS	Executes queued jobs in batches with no user interaction.
Multiprogramming OS	Keeps several jobs in memory so the CPU is never idle.
Multitasking / Time-sharing OS	Gives each user/task a small CPU time slice for interactivity.
Multiprocessing OS	Uses two or more CPUs to execute processes in parallel.
Distributed OS	Manages a group of networked, independent computers as one system.
Network OS	Provides resource and file sharing across autonomous machines.
Real-Time OS	Guarantees task completion within strict deadlines.
Embedded OS	Runs on dedicated hardware with limited memory and power.

Each type is optimised for a different goal – throughput (batch), interactivity (time-sharing), parallelism (multiprocessing), transparency (distributed) or determinism (real-time).

Concept Primer — CPU Scheduling

Read this before the CO1 long-answer scheduling problems

Before solving individual scheduling problems, it helps to understand the whole picture. This primer summarises the key ideas that every scheduling question in the exam builds upon.

What is CPU scheduling?

> **DEFINITION**
>
> **CPU scheduling** is the activity of the operating system that decides **which ready process is given the CPU next, and for how long**. It is the basis of multiprogramming: by switching the CPU among processes, the OS keeps the processor busy and makes the computer more productive.

The scheduling cycle

Process execution alternates between a **CPU burst** (computation) and an **I/O burst** (waiting for input/output). The **short-term scheduler** selects a process from the ready queue whenever the CPU becomes free, and the **dispatcher** then performs the context switch to hand the CPU to that process.

Pre-emptive vs non-preemptive scheduling

Type	Meaning	Examples
Non-preemptive	Once a process gets the CPU it keeps it until it finishes or blocks for I/O.	FCFS, SJF, Priority (basic)
Pre-emptive	The OS can take the CPU away from a running process (e.g. on a higher-priority arrival or a timer interrupt).	Round Robin, SRTF, EDF

Key terms and formulas (used in every numerical)

- **Arrival Time (AT)** – when the process enters the ready queue.
- **Burst Time (BT)** – the CPU time the process needs.
- **Completion Time (CT)** – when the process finishes.
- **Turnaround Time (TAT) = CT – AT.**
- **Waiting Time (WT) = TAT – BT.**
- **Response Time = (time of first CPU allocation) – AT.**

> **EXAM TIP**
>
> Memorise the three-line formula block (TAT = CT–AT, WT = TAT–BT, and the averages). Every scheduling numerical is solved by filling a table with exactly these columns.

Master comparison of scheduling algorithms

Algorithm	Preemptive?	Selection basis	Main advantage	Main drawback
FCFS	No	Arrival order	Simple, fair	Convoy effect
SJF	No	Shortest burst	Min. avg waiting	Starvation; needs burst estimate
SRTF	Yes	Shortest remaining	Better response than SJF	Starvation; overhead

Priority	Either	Highest priority	Flexible / urgent tasks first	Starvation (fixed by aging)
Round Robin	Yes	Time quantum, circular	Fair, good response time	Overhead if quantum too small
MLFQ	Yes	Feedback across queues	Adapts to job behaviour	Complex to tune

Concept Primer — Memory Management

Read this before the CO1 memory-allocation and CO2 paging problems

Why memory management?

> **DEFINITION**
>
> **Memory management** is the OS function that tracks every location in main memory (whether free or allocated), decides which processes get memory and how much, and allocates/de-allocates space as processes come and go – while keeping processes protected from one another.

Logical vs physical address

- **Logical (virtual) address** – generated by the CPU as a program runs; what the program 'sees'.
- **Physical address** – the actual location in RAM.
- The **Memory Management Unit (MMU)** translates logical addresses to physical addresses at run time.

Two families of allocation

Family	Idea	Techniques	Main problem
Contiguous	Each process occupies one contiguous block of memory	Fixed/variable partitions; First/Best/Worst-Fit	External fragmentation
Non-contiguous	A process may be scattered across memory	Paging, Segmentation	Table overhead

Placement strategies for contiguous allocation

- **First-Fit** – use the first hole big enough (fast).
- **Best-Fit** – use the smallest hole that fits (least leftover, but many tiny fragments).
- **Worst-Fit** – use the largest hole (leaves big leftovers, often the poorest utilisation).

Fragmentation

- **Internal fragmentation** – wasted space *inside* an allocated block (e.g. the last page).
- **External fragmentation** – enough total free memory exists, but it is split into holes too small to use. Reduced by **compaction** or by paging.

> **EXAM TIP**
>
> For a fit problem, always redraw the block list after each allocation showing the remaining sizes. The examiner follows your remaining-size column to award step marks.

CO1 — Long Answer Questions

CPU scheduling & memory-management theory with fully worked numerical solutions

Q1. Distinguish the major CPU-scheduling criteria used to evaluate the performance of scheduling algorithms.

Concept: *Scheduling performance metrics* | ***Key terms:*** *utilisation, throughput, waiting/turnaround/response time*

CPU scheduling is the basis of multiprogrammed operating systems. Different scheduling algorithms are compared using well-defined **performance criteria**. Some should be *maximised* and others *minimised*.

Criteria to be MAXIMISED

- **CPU utilisation** – the percentage of time the CPU is kept busy. Ideally as close to 100% as possible (typically 40–90% in real systems).
- **Throughput** – the number of processes completed per unit of time. Higher throughput means more work done.

Criteria to be MINIMISED

- **Turnaround Time (TAT)** – total time from submission of a process to its completion. **TAT = Completion Time** – **Arrival Time.** It includes waiting, execution and I/O time.
- **Waiting Time (WT)** – the total time a process spends waiting in the ready queue. **WT = Turnaround Time** – **Burst Time.**
- **Response Time** – the time from submission until the process produces its *first* response (not full completion). Critical for interactive and real-time systems.

Other considerations

- **Fairness** – every process gets a reasonable share of the CPU.
- **Freedom from starvation** – no process waits indefinitely.

A good scheduling algorithm maximises CPU utilisation and throughput while minimising waiting, turnaround and response times. In a real-time system, low and bounded response time is the most important criterion because meeting deadlines outweighs raw throughput.

> **EXAM TIP**
> Always write the two formulas – TAT = CT – AT and WT = TAT – BT – even in a theory answer. They are frequently worth a mark each and are reused in every numerical.

Q2. Calculate the total number of page faults using FIFO with 3 frames. String: 1, 2, 3, 1, 4, 5, 2, 1, 2, 3.

Concept: *FIFO page replacement* | ***Key terms:*** *oldest-in-first-out, page fault, hit ratio*

> **DEFINITION**
> **Page replacement** is needed in demand paging when a referenced page is not in memory (a **page fault**) and all frames are full, so a victim page must be evicted. Under the **FIFO (First-In-First-Out)** policy the victim is the page that has been in memory the **longest** (the oldest), regardless of how often it is used.

Method

1. Bring a page in on a fault; if frames are full, evict the oldest (front of the FIFO queue).
2. Every reference to a page already present is a **hit** (no fault).

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **FIFO – evict the OLDEST loaded page**.

Reference →	1	2	3	1	4	5	2	1	2	3
Frame 1	**1**	**1**	**1**	**1**	**4**	**4**	**4**	**1**	**1**	**1**
Frame 2		**2**	**2**	**2**	**2**	**5**	**5**	**5**	**5**	**3**
Frame 3			**3**	**3**	**3**	**3**	**2**	**2**	**2**	**2**
Fault / Hit	**F**	**F**	**F**	**H**	**F**	**F**	**F**	**F**	**H**	**F**
Evicted	–	–	–	–	1	2	3	4	–	5

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Result

Total references = 10; Page faults = 8; Hits = 2.
Hit ratio = hits / total = 2/10 = 0.2; Fault ratio = 8/10 = 0.8.

*FIFO is simple but can suffer from **Belady's anomaly** – increasing the number of frames may sometimes increase the number of page faults.*

Q3. Solve using Shortest Job First (SJF). Draw the Gantt chart and calculate waiting and turnaround time.

Concept: *Shortest Job First (non-preemptive)* | ***Key terms:*** *shortest burst next, Gantt chart*

> DEFINITION
> **Shortest Job First (SJF)** selects, from the processes that have arrived and are waiting, the one with the **smallest CPU burst** to execute next. In its non-preemptive form, once a process starts it runs to completion. SJF is **provably optimal** for minimising average waiting time.

Given data

Process	Arrival Time	Burst Time
P1	0	7
P2	1	4
P3	2	1
P4	3	3

Step-by-step decision table — how the CPU is assigned at each moment

Time	Ready queue (burst left)	Chosen	Reason	Runs
t = 0	P1(7)	P1	shortest burst	0 → 7
t = 7	P2(4); P3(1); P4(3)	P3	shortest burst	7 → 8
t = 8	P2(4); P4(3)	P4	shortest burst	8 → 11
t = 11	P2(4)	P2	shortest burst	11 → 15

Gantt chart



Timeline: 0 → 7 → 8 → 11 → 15

Calculation (TAT = CT – AT, WT = TAT – BT)

Process	AT	BT	Completion	Turnaround	Waiting
P1	0	7	7	7	0
P3	2	1	8	6	5
P4	3	3	11	8	5
P2	1	4	15	14	10
Average				**8.75**	**5.0**

Average Turnaround Time = 35/4 = 8.75 units. Average Waiting Time = 20/4 = 5.0 units.

Advantages & disadvantages

- **+** Minimum average waiting time; good for batch systems.
- – Requires knowing burst times in advance; long jobs may **starve**.

Q4. Compute the execution order using Priority Scheduling (lower value = higher priority). Draw the Gantt chart.

Concept: *Priority scheduling (non-preemptive)* | ***Key terms:*** *priority order, starvation, aging*

> DEFINITION
>
> In **Priority Scheduling** each process is given a priority number and the CPU is allocated to the process with the **highest priority** (here, the **lowest numerical value**). Ties are broken by arrival order. Shown below in non-preemptive form.

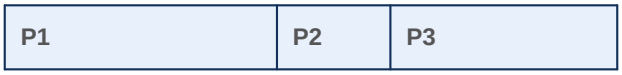
Given data

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	2	1
P3	2	5	2

Step-by-step decision table — how the CPU is assigned at each moment

Time	Ready queue (burst, priority)	Chosen	Reason	Runs
t = 0	P1(6, pri 3)	P1	lowest priority number	0 → 6
t = 6	P2(2, pri 1); P3(5, pri 2)	P2	lowest priority number	6 → 8
t = 8	P3(5, pri 2)	P3	lowest priority number	8 → 13

Gantt chart



Timeline: 0 → 6 → 8 → 13

Calculation

Process	AT	BT	Priority	Completion	Turnaround	Waiting
P1	0	6	3	6	6	0

P2	1	2	1	8	7	5
P3	2	5	2	13	11	6
Average					**8.0**	**3.67**

Average Turnaround Time = 24/3 = 8.0 units. Average Waiting Time = 11/3 ≈ 3.67 units.

Problem – starvation: *low-priority processes may never run. The solution is* ***aging*** *– gradually raising the priority of processes that have waited a long time.*

Q5. Execute FIFO page replacement with 3 frames. String: 4, 1, 2, 4, 3, 5, 2, 1, 4, 3.

Concept: *FIFO page replacement* | ***Key terms:*** *oldest-out*

Rule: on a fault with full frames, evict the oldest resident page.

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **FIFO – evict the OLDEST loaded page**.

Reference →	4	1	2	4	3	5	2	1	4	3
Frame 1	**4**	**4**	**4**	**4**	**3**	**3**	**3**	**3**	**4**	**4**
Frame 2		**1**	**1**	**1**	**1**	**5**	**5**	**5**	**5**	**3**
Frame 3			**2**	**2**	**2**	**2**	**2**	**1**	**1**	**1**
Fault / Hit	**F**	**F**	**F**	**H**	**F**	**F**	**H**	**F**	**F**	**F**
Evicted	–	–	–	–	4	1	–	2	3	5

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Total references = 10; Page faults = 8; Hits = 2; Hit ratio = 0.2.

Q6. Construct the allocation using First-Fit and Worst-Fit. Blocks: 150, 350, 200, 500, 100 KB; Requests: 120, 275, 180, 425 KB.

Concept: *Contiguous allocation strategies* | ***Key terms:*** *first-fit, worst-fit, external fragmentation*

> **DEFINITION**
>
> In **contiguous memory allocation**, a process must be placed in a single contiguous block (hole). A **placement algorithm** decides which free hole to use. **First-Fit** picks the first hole that is large enough; **Worst-Fit** picks the *largest* hole; **Best-Fit** picks the smallest hole that fits.

Step-by-step allocation table — First-Fit (choose the first block large enough)

Request	Blocks available (KB)	Block chosen	Leftover	Blocks after (KB)
120 KB	150, 350, 200, 500, 100	150 KB	30 KB	30, 350, 200, 500, 100
275 KB	30, 350, 200, 500, 100	350 KB	75 KB	30, 75, 200, 500, 100
180 KB	30, 75, 200, 500, 100	200 KB	20 KB	30, 75, 20, 500, 100
425 KB	30, 75, 20, 500, 100	500 KB	75 KB	30, 75, 20, 75, 100

First-Fit result: all requests were allocated successfully; spare blocks remain free.

Step-by-step allocation table — Worst-Fit (choose the largest block available)

Request	Blocks available (KB)	Block chosen	Leftover	Blocks after (KB)

120 KB	150, 350, 200, 500, 100	500 KB	380 KB	150, 350, 200, 380, 100
275 KB	150, 350, 200, 380, 100	500 KB	105 KB	150, 350, 200, 105, 100
180 KB	150, 350, 200, 105, 100	350 KB	170 KB	150, 170, 200, 105, 100
425 KB	150, 170, 200, 105, 100	none – fails	–	150, 170, 200, 105, 100

Worst-Fit result: the 425 KB request cannot be placed – the total free memory is enough, but no single hole is large enough. This is external fragmentation.

First-Fit is fast and here succeeds for every request; Worst-Fit intentionally leaves large leftovers but failed the biggest request. Best-Fit minimises leftover but tends to create many tiny unusable holes.

Q7. Calculate page faults using LRU with 3 frames. String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2.

Concept: *LRU page replacement* | ***Key terms:*** *least-recently-used, locality*

> DEFINITION
>
> **LRU (Least Recently Used)** replaces the page that has **not been used for the longest time**. It approximates the optimal algorithm by assuming that pages used recently will be used again soon (the principle of **locality of reference**).

Method: on a fault with full frames, evict the page whose *last use* is furthest in the past.

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **LRU – evict the LEAST-RECENTLY-USED page**.

Reference →	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2		0	0	0	0	0	0	0	0	3	3	3	3
Frame 3			1	1	1	3	3	3	2	2	2	2	2
Fault / Hit	F	F	F	F	H	F	H	F	F	F	F	H	H
Evicted	–	–	–	7	–	1	–	2	3	0	4	–	–

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Total references = 13; Page faults = 9; Hits = 4; Hit ratio = 4/13 ≈ 0.31.

LRU does not suffer from Belady's anomaly (it is a 'stack algorithm'), but exact LRU is costly to implement in hardware, so approximations (reference bits, second-chance) are used in practice.

Q8. Determine execution order and average waiting time using FCFS. Draw the Gantt chart.

Concept: *First-Come First-Served* | ***Key terms:*** *arrival order, convoy effect*

> DEFINITION
>
> **First-Come First-Served (FCFS)** is the simplest scheduling algorithm: the process that requests the CPU **first** is allocated the CPU first. It is non-preemptive and implemented with a simple FIFO queue.

Given data

Process	Arrival Time	Burst Time
P1	0	5

P2	1	3
P3	2	6
P4	4	2

Step-by-step decision table — how the CPU is assigned at each moment

Time	Ready queue (burst left)	Chosen	Reason	Runs
t = 0	P1(5)	P1	earliest arrival	0 → 5
t = 5	P2(3); P3(6); P4(2)	P2	earliest arrival	5 → 8
t = 8	P3(6); P4(2)	P3	earliest arrival	8 → 14
t = 14	P4(2)	P4	earliest arrival	14 → 16

Gantt chart (served strictly in arrival order)

P1	P2	P3	P4

Timeline: 0 → 5 → 8 → 14 → 16

Calculation

Process	AT	BT	Completion	Turnaround	Waiting
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	6	14	12	6
P4	4	2	16	12	10
Average				**9.0**	**5.0**

Average Waiting Time = 20/4 = 5.0 units. Average Turnaround Time = 36/4 = 9.0 units.

Convoy effect: *a long process at the front makes short processes wait a long time behind it, raising the average waiting time. FCFS is simple and fair but gives poor average performance.*

Q9. Demonstrate the allocation using Best-Fit. Blocks: 100, 500, 200, 300, 600 KB; Requests: 212, 417, 112, 426 KB.

Concept: *Best-Fit allocation* | ***Key terms:*** *smallest sufficient hole*

Best-Fit allocates the **smallest** free block that is still large enough for the request, aiming to leave the smallest possible leftover.

Step-by-step allocation table — Best-Fit (choose the smallest block that fits)

Request	Blocks available (KB)	Block chosen	Leftover	Blocks after (KB)
212 KB	100, 500, 200, 300, 600	300 KB	88 KB	100, 500, 200, 88, 600
417 KB	100, 500, 200, 88, 600	500 KB	83 KB	100, 83, 200, 88, 600
112 KB	100, 83, 200, 88, 600	200 KB	88 KB	100, 83, 88, 88, 600
426 KB	100, 83, 88, 88, 600	600 KB	174 KB	100, 83, 88, 88, 174

Best-Fit result: all requests were allocated successfully; spare blocks remain free.

Best-Fit result: all four requests are satisfied; the 100 KB block remains free. Best-Fit uses memory tightly but the many small leftovers (88, 83, 88 KB) can accumulate as external fragmentation over time.

Q10. Demonstrate the working of Multi-Level Feedback Queue (MLFQ) scheduling and how queue priorities influence execution.

Concept: *MLFQ* | ***Key terms:*** *multiple queues, feedback, aging*

> **DEFINITION**
>
> **Multi-Level Feedback Queue (MLFQ)** scheduling uses **several ready queues** of different priorities, each with its own scheduling policy and time quantum, and allows a process to **move between queues** based on its observed CPU-burst behaviour ('feedback').

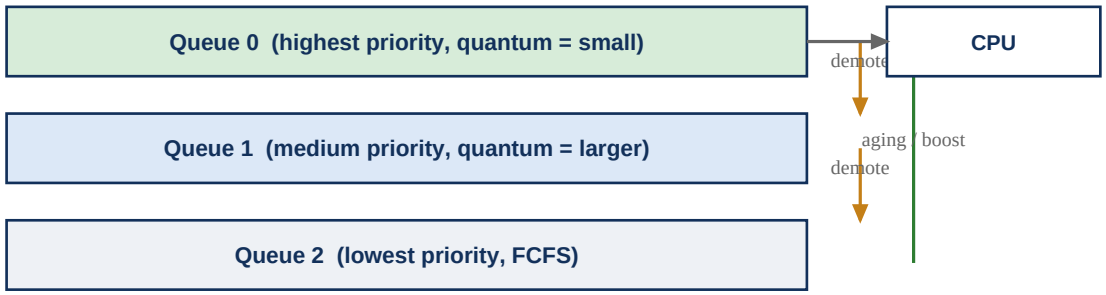


Fig: MLFQ – CPU-bound jobs are demoted to lower queues; aging promotes long-waiting jobs.

Working rules

1. A new process enters the **highest-priority** queue, which has the **smallest** time quantum.
2. If a process uses its **entire quantum** (CPU-bound), it is **demoted** to a lower-priority queue with a larger quantum.
3. If a process **gives up the CPU early** (I/O-bound / interactive), it **stays** at its high priority.
4. A higher-priority queue is always served before a lower one, and can **pre-empt** it.
5. **Aging** periodically boosts long-waiting processes to a higher queue to prevent starvation.

Effect of queue priorities: short and interactive jobs finish quickly at high priority, while long CPU-bound jobs sink to lower queues and run when the CPU is otherwise idle. MLFQ thus approximates SJF without knowing burst times in advance, giving good response time and throughput together.

Q11. Illustrate real-time scheduling and state how task timing requirements influence scheduling decisions.

Concept: *Real-time scheduling* | ***Key terms:*** *deadlines, periodic tasks, RMS, EDF*

> **DEFINITION**
>
> **Real-time scheduling** orders task execution so that every task meets its **deadline**. Tasks are usually **periodic** and described by three parameters: period (T), worst-case execution time (C) and deadline (D).

Common real-time scheduling algorithms

- **Rate-Monotonic Scheduling (RMS)** – *static* priority: the **shorter the period, the higher the priority**. Simple and predictable.

- **Earliest-Deadline-First (EDF)** – *dynamic* priority: the task with the **nearest deadline** runs next. More flexible, higher achievable utilisation.
- Both are usually **pre-emptive** so an urgent task can interrupt a less urgent one.

How timing requirements influence decisions

- **Deadline** determines urgency – a nearer deadline demands earlier execution.
- **Period** sets how often a task must run and (in RMS) fixes its priority.
- **Criticality** (hard vs soft) decides whether a miss is tolerable.
- A **schedulability test** checks in advance whether the task set is feasible.

Unlike ordinary scheduling that optimises average throughput, real-time scheduling is driven entirely by timing parameters to guarantee deadlines.

Q12. Examine the principal CPU-scheduling criteria used to assess the efficiency of a scheduling algorithm.

Concept: *Scheduling metrics (applied)* | ***Key terms:*** *utilisation, throughput, WT, TAT, response*

The efficiency of any scheduling algorithm is assessed using the same principal criteria used to compare algorithms (see also Q1):

Criterion	Meaning	Goal
CPU utilisation	% of time the CPU is busy	Maximise
Throughput	Processes finished per unit time	Maximise
Turnaround time	Completion – Arrival	Minimise
Waiting time	Time spent in ready queue	Minimise
Response time	Submission → first response	Minimise
Fairness	Equitable CPU share, no starvation	Maximise

For batch systems throughput and turnaround dominate; for interactive systems response time dominates; for real-time systems a low and bounded response time is the key measure because deadlines must be met.

Q13. Illustrate Round Robin scheduling and discuss the effect of time quantum on response and context-switching overhead.

Concept: *Round Robin* | ***Key terms:*** *time quantum, context switch, response time*

> **DEFINITION**
>
> **Round Robin (RR)** is a pre-emptive scheduling algorithm designed for time-sharing systems. Each ready process is given a fixed **time quantum (time slice)** in circular FIFO order; if it does not finish within the quantum it is pre-empted and moved to the **back** of the ready queue.

Illustrative example (quantum = 4)

Processes P1(BT 8), P2(BT 4), P3(BT 5), all arriving at t=0.



Timeline: 0 → 4 → 8 → 12 → 16 → 17

Each process gets 4 units in turn; P2 finishes early, P1 and P3 return for their remaining bursts.

Effect of the time quantum

- **Very small quantum** → excellent responsiveness, but **frequent context switches** waste CPU time on switching overhead.
- **Very large quantum** → RR degenerates into **FCFS**, giving poor response time.
- The quantum should be **slightly larger than a typical CPU burst** (often 10–100 ms) to balance responsiveness against overhead.

RR is starvation-free and fair, which makes it ideal for interactive/time-sharing systems.

> **EXAM TIP**
> If asked about the quantum, always mention the two extremes: 'too small = high context-switch overhead' and 'too large = behaves like FCFS'. This contrast usually carries the marks.

Q14. Summarise the factors considered when selecting a CPU-scheduling algorithm for a real-time system.

Concept: *Algorithm selection for RTOS* | ***Key terms:*** *predictability, pre-emption, overhead*

For a real-time system, deadlines matter more than average speed, so the following factors guide the choice of scheduling algorithm:

1. **Predictability / determinism** – the worst-case timing must be bounded and analysable.
2. **Deadline and priority support** – the algorithm must honour task priorities and deadlines (e.g. RMS, EDF).
3. **Pre-emptiveness** – an urgent high-priority task must be able to interrupt a running task.
4. **Low scheduling and context-switch overhead** – the scheduler itself must be fast.
5. **Schedulability** – a mathematical test should confirm all deadlines can be met.
6. **Priority-inversion handling** – support for priority inheritance/ceiling protocols.
7. **Nature of tasks** – periodic vs aperiodic, and hard vs soft real-time requirements.

Q15. Apply response, waiting and turnaround time to assess the suitability of a scheduling approach for time-sensitive processes.

Concept: *Applying metrics to time-sensitive work* | ***Key terms:*** *response time priority*

For **time-sensitive (real-time) processes**, the three timing metrics are interpreted as follows:

- **Response time** (submission → first execution) is the **most critical**: a time-sensitive task must begin reacting quickly, so a small, bounded response time is essential.
- **Waiting time** (idle time in the ready queue) must be minimised so an urgent task is not delayed behind long jobs.
- **Turnaround time** (submission → completion) shows whether the whole task finishes before its deadline.

Assessment

A pre-emptive priority or EDF approach keeps response and waiting times low for high-priority tasks, so it is well suited to time-sensitive processes. By contrast, FCFS is unsuitable: its convoy effect inflates all three metrics, and a long job can make an urgent task miss its deadline.

Q16. Apply MLFQ scheduling to a set of processes and show how processes move between different priority queues.

Concept: *MLFQ movement* | ***Key terms:*** *promotion, demotion, aging*

Consider three queues: **Q0** (quantum 4, highest priority), **Q1** (quantum 8, medium), and **Q2** (FCFS, lowest). Take processes A (CPU-bound, long burst) and B (I/O-bound, short bursts).

Step	Action	Result / movement
1	A and B enter Q0	Both start at highest priority
2	A runs, uses full quantum (4)	A demoted to Q1
3	B runs, blocks for I/O early	B stays in Q0 (interactive)
4	A runs in Q1, uses full quantum (8)	A demoted to Q2
5	B returns from I/O	B re-enters Q0, runs quickly
6	A waits long in Q2	Aging boosts A back up

Thus the CPU-bound process A gradually sinks Q0→Q1→Q2, while the I/O-bound process B stays at high priority for fast response. Queue level effectively becomes the execution priority, and aging prevents starvation of demoted jobs.

Q17. Discuss the main features of multiprocessor scheduling in a real-time operating system.

Concept: *Multiprocessor scheduling* | ***Key terms:*** *load balancing, affinity, partitioning*

> DEFINITION
>
> **Multiprocessor scheduling** deals with assigning tasks to **two or more CPUs** so that the load is balanced and, in an RTOS, all deadlines are still met.

Main features

- **Load balancing** – distributing tasks so no CPU is overloaded while others are idle (push/pull migration).
- **Partitioned vs global scheduling** – each task is fixed to one CPU (partitioned), or all CPUs share a single global ready queue (global).
- **Processor affinity** – keeping a task on the same CPU to reuse its cache contents and reduce migration cost.
- **SMP vs AMP** – in **symmetric** multiprocessing every CPU runs the OS and shares the load; in **asymmetric** multiprocessing one master CPU controls the others.
- **Synchronisation** – careful locking to protect shared data and to bound priority inversion across cores.
- **Schedulability guarantees** – real-time multiprocessor scheduling must still prove that deadlines are met on every core.

Q18. Implement Rate-Monotonic Scheduling (RMS) for periodic tasks and assign priorities according to task periods.

Concept: *RMS* | ***Key terms:*** *fixed priority, shorter period = higher priority, utilisation bound*

> DEFINITION

> **Rate-Monotonic Scheduling (RMS)** is a static-priority, pre-emptive algorithm for **periodic** real-time tasks. It assigns priorities by **rate**: the task with the **shortest period** (highest rate) gets the **highest priority**.

Example task set

(The paper says 'the following periodic tasks' without listing them, so a standard set is used to demonstrate the method.)

Task	Period T (ms)	Execution C (ms)	Priority (by RMS)
T1	20	4	Highest (shortest period)
T2	40	8	Medium
T3	80	16	Lowest (longest period)

Schedulability test (Liu & Layland)

CPU utilisation: $U = \Sigma (C_i / T_i) = 4/20 + 8/40 + 16/80 = 0.2 + 0.2 + 0.2 = \mathbf{0.6}$.

The utilisation bound for n tasks is $U \leq n(2^{1/n} - 1)$. For n = 3: $3(2^{1/3} - 1) \approx \mathbf{0.779}$.

Since $0.6 \leq 0.779$, the task set is schedulable under RMS and every deadline is guaranteed. T1 (shortest period) can pre-empt T2 and T3; T3 runs only when T1 and T2 are not ready.

*RMS is **optimal among fixed-priority** algorithms: if any fixed-priority assignment can meet all deadlines, the rate-monotonic assignment can too.*

Concept Primer — Virtual Memory & Paging

Read this before the CO2 short and long answers

What is virtual memory?

> **DEFINITION**
>
> **Virtual memory** is a technique that lets a process execute even if it is **not entirely in main memory**. Only the pages currently needed are kept in RAM; the rest stay on disk (the backing store) and are brought in on demand. This gives programmers the illusion of a very large, private memory.

How paging supports virtual memory

- Logical memory is split into fixed-size **pages**; physical memory into equal-size **frames**.
- A **page table** per process maps page numbers to frame numbers.
- The **valid/invalid bit** in each entry tells whether the page is currently in memory.
- A reference to a page that is not in memory causes a **page fault**, and the OS loads it (**demand paging**).

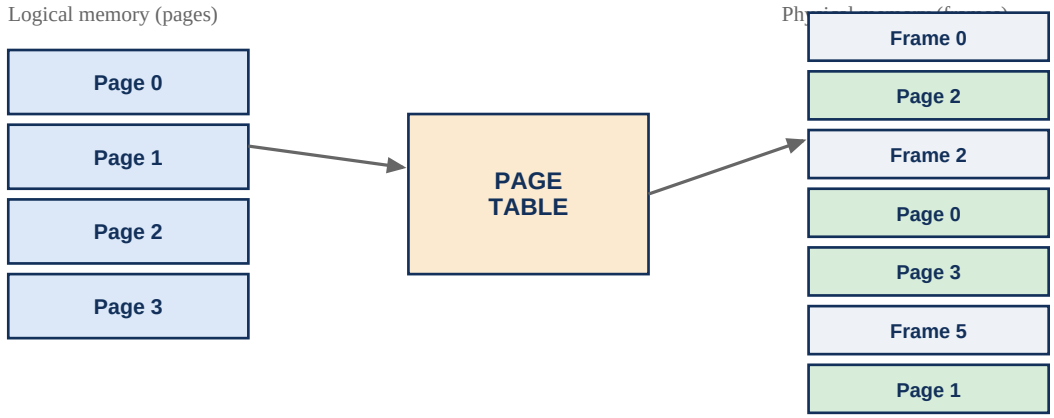


Fig: Pages are mapped to scattered frames; not all pages need be resident at once.

Address translation & the TLB

Every logical address is **(page number, offset)**. The page number indexes the page table to get a **frame number**; the offset is unchanged. To avoid an extra memory access per reference, a small fast cache called the **Translation Lookaside Buffer (TLB)** stores recent page→frame mappings.

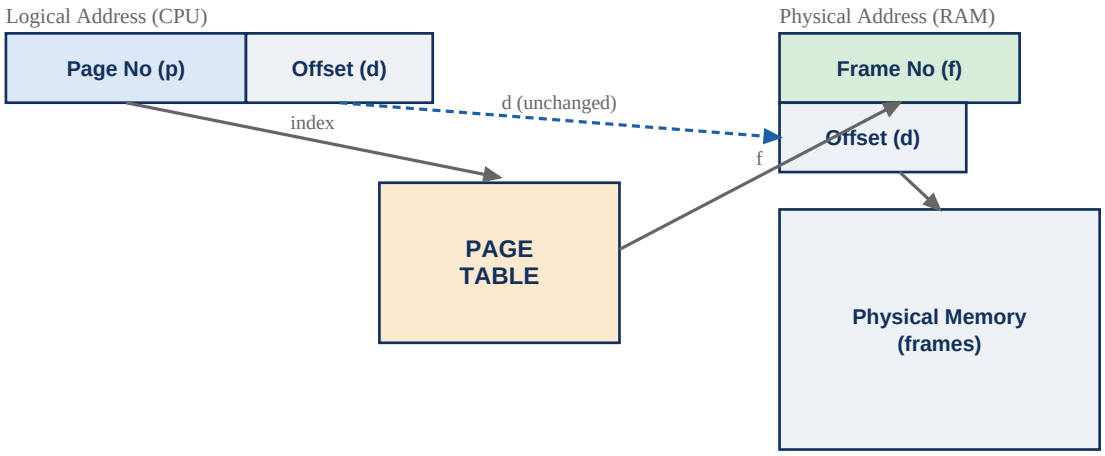


Fig: Logical-to-physical address translation using the page table.

Page-replacement algorithms (overview)

When a page fault occurs and memory is full, a **victim page** must be chosen. The algorithm used decides which page to evict:

Algorithm	Victim chosen	Property
FIFO	Oldest page in memory	Simple; can show Belady's anomaly
LRU	Least recently used page	Good; no Belady anomaly (stack algo)
Optimal (OPT)	Page not used for longest in the future	Fewest faults; not implementable

Key formulas

- **Hit ratio = number of hits / total references.**
- **Page-fault ratio = number of page faults / total references.**
- **EAT (with TLB) = $h(t_{TLB}+t_{mem}) + (1-h)(t_{TLB}+2t_{mem})$.**

Thrashing (the danger)

If too many processes share too few frames, the system spends almost all its time paging in and out instead of computing – this is **thrashing**. It is controlled with the **working-set model** and by limiting the degree of multiprogramming.

> **EXAM TIP**
> In every page-replacement numerical, draw the frame table column-by-column, mark each reference as Hit or Fault, and count the faults at the end. State the hit ratio to earn the final mark.

CO2 — Short Answer Questions

Virtual memory, address spaces, RTOS tasks, process & memory APIs

Q1. Identify the purpose of the Translation Lookaside Buffer (TLB) in virtual-memory address translation.

Concept: *TLB* | ***Key terms:*** *cache of page-table entries, faster translation*

> **DEFINITION**
>
> The **Translation Lookaside Buffer (TLB)** is a small, fast hardware cache inside the Memory Management Unit (MMU) that stores recently used **page-number** → **frame-number** mappings.

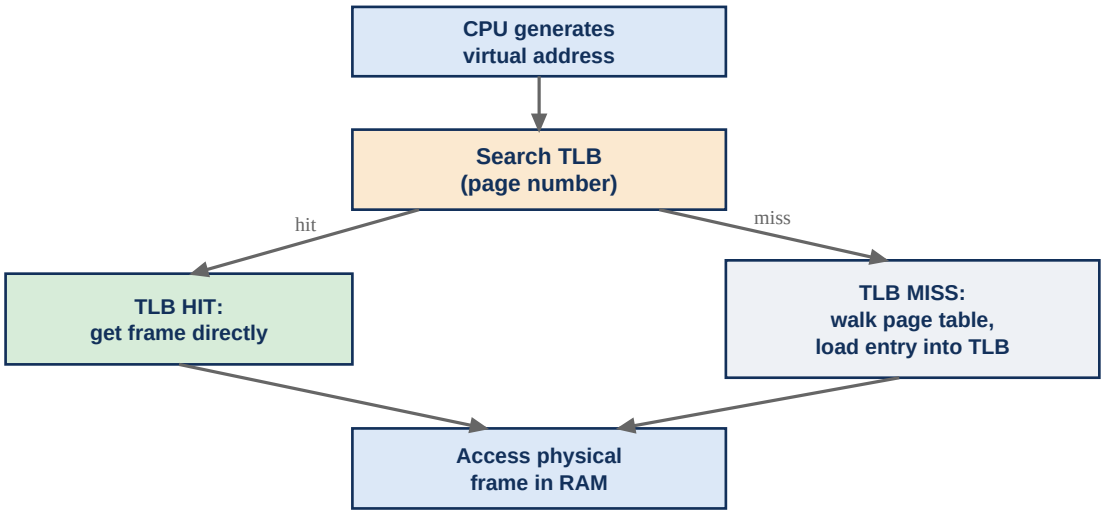


Fig: On a TLB hit the frame is found instantly; a miss requires a page-table walk.

Purpose

- **Speeds up address translation** – a **TLB hit** gives the physical frame in one step, avoiding a separate memory access to the page table.
- On a **TLB miss** the page table in main memory is walked and the mapping is loaded into the TLB.
- Exploits **locality of reference**: most accesses hit the same few pages, so the hit ratio is high.
- Removes the extra memory access that pure paging would otherwise add to every instruction.

Effective Access Time (EAT) = h×(TLB+mem) + (1–h)×(TLB+2×mem), where h is the hit ratio; a high h keeps EAT close to a single memory access.

Q2. Relate the Memory API to memory allocation and management in a virtualised memory environment.

Concept: *Memory API* | ***Key terms:*** *malloc/free, brk/sbrk, mmap*

> **DEFINITION**
>
> The **Memory API** is the set of functions and system calls that programs use to request and release memory dynamically at run time.

Main calls

- **malloc() / calloc() / realloc() / free()** – library functions for allocating and freeing heap memory.
- **brk() / sbrk()** – system calls that grow or shrink the program's heap (data segment).
- **mmap() / munmap()** – map files or anonymous memory regions into the address space.

Relation to virtualised memory

In a virtualised environment these calls allocate space within the process's virtual address space. The OS then maps the requested virtual pages to physical frames on demand (demand paging), completely transparently to the program. The API therefore hides physical-memory details and supports protection and isolation, letting each process behave as if it had a large private memory.

Q3. State the significance of real-time programming languages in the development of RTOS applications.

Concept: *Real-time languages* | ***Key terms:*** *predictability, concurrency, timing constructs*

Ordinary general-purpose languages cannot guarantee timing behaviour. **Real-time programming languages** are significant because they provide features essential for meeting deadlines:

- **Deterministic, predictable execution** – bounded timing and controllable (or absent) garbage collection.
- **Built-in concurrency and task/priority constructs** that map directly to RTOS tasks (e.g. Ada tasks, Real-Time Java threads).
- **Direct timing and interrupt handling** to react precisely to external events.
- **Reliability and safety features** – strong typing, exception handling and formal verification support, needed in safety-critical systems.

Examples: Ada, C/C++ with RTOS libraries, and Real-Time Java. These languages let developers express and guarantee timing behaviour that a general-purpose language cannot.

Q4. Characterise an RTOS task and identify two characteristics that distinguish a real-time task from a conventional process.

Concept: *RTOS task* | ***Key terms:*** *timing attributes, lightweight*

> **DEFINITION**
> An **RTOS task** is a schedulable unit of execution associated with **timing attributes** – a period, a deadline, a priority and a worst-case execution time (WCET).

Two distinguishing characteristics

1. **Explicit timing constraints (deadline):** a real-time task must complete before a defined **deadline**, whereas a conventional process simply runs to completion whenever the CPU is available, with no deadline.
2. **Priority-driven, pre-emptive scheduling:** real-time tasks are scheduled by strict priority or deadline and are usually **lightweight** (often sharing one address space), while conventional processes are scheduled for fairness/throughput in separate, protected address spaces.

Other real-time task features include **periodicity** and a known, **bounded WCET** that enables schedulability analysis.

Q5. Outline the concept of an address space and its role in memory virtualisation.

Concept: *Address space* | ***Key terms:*** *logical view, isolation*

> **DEFINITION**
> An **address space** is the range of **logical (virtual) addresses** that a process may use. It typically contains the program's **code (text), data, heap and stack**.

Role in memory virtualisation

- Each process is given its **own private virtual address space**, usually starting at address 0, which the OS/MMU maps to physical memory.
- Provides the **illusion** that every process has a large, private, contiguous memory of its own.
- Gives **isolation and protection** – one process cannot read or corrupt another's memory.
- Decouples the **logical layout from the physical layout**, enabling demand paging, sharing and relocation.

Thus the address space is the central abstraction that makes memory virtualisation possible.

Q6. Identify two important characteristics of RTOS tasks that support time-constrained execution.

Concept: *RTOS task characteristics* | ***Key terms:*** *deadline, priority, periodicity, bounded WCET*

1. **Deadline and periodicity:** each task has a defined **deadline** (and often a fixed period), so the scheduler knows exactly when the task must run and finish.
2. **Priority and pre-emptability:** tasks carry priorities and can pre-empt or be pre-empted, allowing an urgent task to run immediately when it becomes ready.

A third supporting characteristic is a **bounded worst-case execution time (WCET)**, which makes it possible to prove mathematically that all deadlines can be met (schedulability analysis).

Q7. Discuss the role of processes and processor states in an Operating System.

Concept: *Process states* | ***Key terms:*** *new, ready, running, waiting, terminated*

> **DEFINITION**
> A **process** is a program in execution. It is represented in the OS by a **Process Control Block (PCB)** holding its process ID, state, program counter, registers, memory limits and open files.

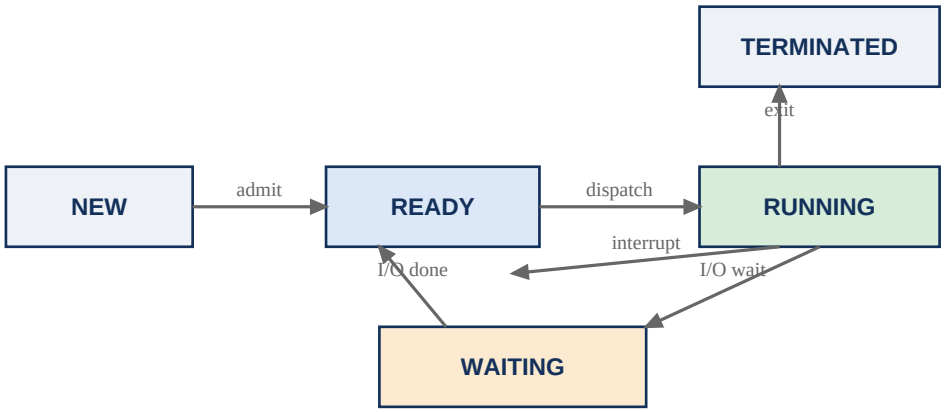


Fig: The five-state process model and the transitions between states.

Process states

- **New** – the process is being created.
- **Ready** – the process is waiting to be assigned to the CPU.
- **Running** – instructions are being executed on the CPU.
- **Waiting / Blocked** – the process is waiting for an I/O completion or an event.
- **Terminated** – the process has finished execution.

Tracking these states lets the OS schedule the CPU correctly, perform context switches and manage resources, which is the foundation of multiprogramming.

Q8. Explain how paging provides efficient utilisation of physical memory.

Concept: *Paging* | ***Key terms:*** *fixed-size pages/frames, no external fragmentation*

> DEFINITION
>
> **Paging** is a memory-management scheme that divides logical memory into fixed-size **pages** and physical memory into equal-size **frames**. A **page table** maps each page to a frame, so a process need not occupy contiguous memory.

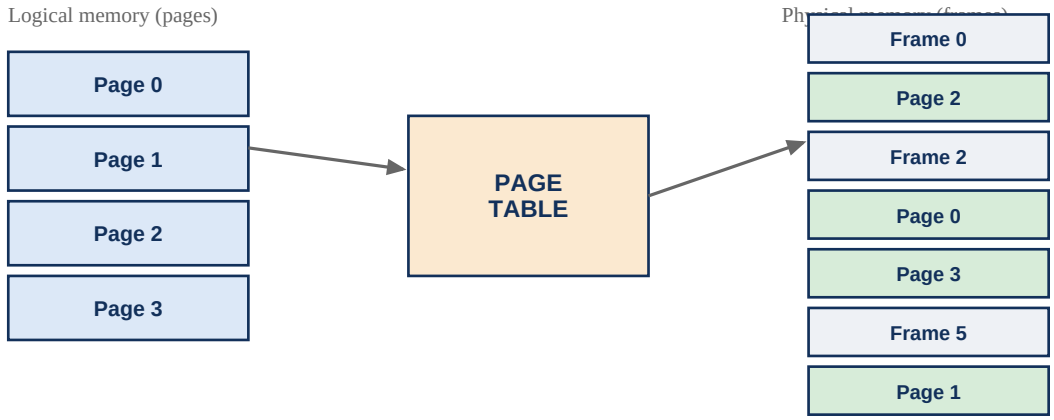


Fig: Logical pages are mapped by the page table to scattered physical frames.

How paging improves memory utilisation

- **Eliminates external fragmentation** – any free frame can hold any page.
- **Non-contiguous allocation** lets scattered free frames be used fully.

- Combined with **demand paging**, only the needed pages are loaded, so **more processes fit** in memory at once.
- The only overhead is small **internal fragmentation** in the last page of a process.

Thus paging gives near-complete use of physical memory with minimal waste.

Q9. Describe the purpose of a Process API in creating and managing processes.

Concept: *Process API* | ***Key terms:*** *fork, exec, wait, exit*

> DEFINITION
>
> The **Process API** is the set of system calls provided by the OS to **create, control and terminate processes**.

Key UNIX/Linux calls

- **fork()** – creates a new (child) process that is an exact copy of the parent; returns twice (0 to the child, child's PID to the parent).
- **exec()** – replaces the current process image with a new program.
- **wait()** – makes the parent block until a child terminates and then collects its exit status.
- **exit()** – terminates a process and returns a status code to the parent.

Together these calls let the OS and applications create, run, synchronise and cleanly terminate processes, supporting the whole process-management subsystem.

CO2 — Long Answer Questions

Page-replacement problems plus virtual-memory, IPC/RPC and segmentation theory

Q1. Perform FIFO page replacement with 4 frames. String: 5, 2, 1, 3, 2, 4, 5, 1, 3, 2, 4.

Concept: *FIFO page replacement* | ***Key terms:*** *4 frames, oldest-out*

Rule: on a fault with all four frames full, evict the page that entered earliest (oldest).

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **FIFO – evict the OLDEST loaded page**.

Reference →	5	2	1	3	2	4	5	1	3	2	4
Frame 1	5	5	5	5	5	4	4	4	4	4	4
Frame 2		2	2	2	2	2	5	5	5	5	5
Frame 3			1	1	1	1	1	1	1	2	2
Frame 4				3	3	3	3	3	3	3	3
Fault / Hit	F	F	F	F	H	F	F	H	H	F	H
Evicted	–	–	–	–	–	5	2	–	–	1	–

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Total references = 11; Page faults = 7; Hits = 4; Hit ratio = 4/11 ≈ 0.36.

Q2. Contrast local Inter-Process Communication (IPC) and Remote Procedure Call (RPC) with respect to communication scope and operation in an RTOS.

Concept: *IPC vs RPC* | ***Key terms:*** *same machine vs across network*

> DEFINITION
>
> **Inter-Process Communication (IPC)** lets processes on the **same machine** exchange data and synchronise. A **Remote Procedure Call (RPC)** lets a process invoke a procedure that executes on a **different machine**, as if it were a local call.

Aspect	Local IPC	Remote Procedure Call (RPC)
Scope	Processes on the same machine	Processes on different machines (network)
Mechanism	Shared memory, pipes, message queues, semaphores	Procedure-call semantics over a network
Data transfer	Direct via kernel or shared memory	Parameters are marshalled and sent as messages
Speed	Fast (no network)	Slower (network latency)
Transparency	Programmer manages communication explicitly	Looks like an ordinary local function call
Failure modes	Few	Network / partial-failure issues

Summary: local IPC coordinates tasks within one node quickly and simply; RPC extends the call model across nodes so a task can use a remote service transparently, at the cost of network latency.

Q3. Perform LRU page replacement with 3 frames. String: 2, 4, 1, 2, 5, 1, 3, 2, 4, 1.

Concept: *LRU page replacement* | ***Key terms:*** *least-recently-used*

Rule: evict the page whose most recent use is furthest in the past.

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **LRU – evict the LEAST-RECENTLY-USED page**.

Reference →	2	4	1	2	5	1	3	2	4	1
Frame 1	2	2	2	2	2	2	3	3	3	1
Frame 2		4	4	4	5	5	5	2	2	2
Frame 3			1	1	1	1	1	1	4	4
Fault / Hit	F	F	F	H	F	H	F	F	F	F
Evicted	–	–	–	–	4	–	2	5	1	3

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Total references = 10; Page faults = 8; Hits = 2; Hit ratio = 0.2.

Q4. Obtain the total number of page faults using Optimal page replacement with 3 frames. String: 7, 2, 5, 3, 2, 4, 2, 1, 5, 2.

Concept: *Optimal (Belady) replacement* | ***Key terms:*** *evict farthest future use*

> DEFINITION
>
> The **Optimal (OPT / Belady) algorithm** replaces the page that will **not be used for the longest time in the future**. It gives the **minimum possible** number of page faults and is used as a benchmark, but it is not implementable in practice because it needs future knowledge.

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **Optimal – evict the page used FARTHEST in the future**.

Reference →	7	2	5	3	2	4	2	1	5	2
Frame 1	7	7	7	3	3	4	4	1	1	1
Frame 2		2	2	2	2	2	2	2	2	2
Frame 3			5	5	5	5	5	5	5	5
Fault / Hit	F	F	F	F	H	F	H	F	H	H
Evicted	–	–	–	7	–	3	–	4	–	–

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Why each Optimal eviction (look-ahead)

- Column 4 (page 3): next use – 7→never, 2→pos 5, 5→pos 9 ⇒ evict **7**.
- Column 6 (page 4): next use – 3→never, 2→pos 7, 5→pos 9 ⇒ evict **3**.
- Column 8 (page 1): next use – 4→never, 2→pos 10, 5→pos 9 ⇒ evict **4**.

Total references = 10; Page faults = 6; Hits = 4. This is the minimum achievable for this string.

Q5. Interpret demand paging and swapping, and indicate how excessive paging activity results in thrashing.

Concept: *Demand paging, swapping, thrashing* | ***Key terms:*** *page fault, working set*

> DEFINITION
>
> **Demand paging** loads a page into memory only when it is first referenced; the reference to an absent page causes a **page fault** that brings the page in. **Swapping** moves whole processes (or pages) between main memory and the backing store (disk) to free space.

How thrashing arises

- When too many processes compete for too few frames, each process's **working set** does not fit in memory.
- Every process keeps faulting and evicting pages that are needed again almost immediately.
- The system then spends nearly all its time on **paging I/O** instead of useful computation, and CPU utilisation collapses – this is **thrashing**.

Control measures

- Use the **working-set model** to give each process enough frames for its active pages.
- Use a **page-fault-frequency (PFF)** scheme to add/remove frames dynamically.
- **Reduce the degree of multiprogramming** (suspend some processes) when the fault rate is too high.

Note: this is a sensitive-balance problem – demand paging and swapping are efficient until memory is over-committed, at which point they degenerate into thrashing.

Q6. Select page-replacement decisions using LRU with 4 frames and calculate the page faults. String: 3, 1, 4, 2, 1, 5, 3, 2, 6, 1, 4.

Concept: *LRU page replacement* | ***Key terms:*** *4 frames*

Rule: evict the least-recently-used page when all four frames are full.

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **LRU – evict the LEAST-RECENTLY-USED page**.

Reference →	3	1	4	2	1	5	3	2	6	1	4
Frame 1	3	3	3	3	3	5	5	5	5	1	1
Frame 2		1	1	1	1	1	1	1	6	6	6
Frame 3			4	4	4	4	3	3	3	3	4
Frame 4				2	2	2	2	2	2	2	2
Fault / Hit	F	F	F	F	H	F	F	H	F	F	F
Evicted	–	–	–	–	–	3	4	–	1	5	3

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Total references = 11; Page faults = 9; Hits = 2; Hit ratio = 2/11 ≈ 0.18.

Q7. Derive the total number of page faults using Optimal page replacement with 3 frames. String: 6, 2, 1, 3, 2, 4, 1, 5, 2, 3.

Concept: *Optimal replacement* | ***Key terms:*** *farthest future use*

Rule: evict the page whose next use is farthest in the future (or not used again).

Step-by-step trace table — frame state after each reference

Read the table left-to-right. Each column is one page reference from the string; the cells below show what each frame holds **after** that reference is processed. Rule: **Optimal – evict the page used FARTHEST in the future**.

Reference →	6	2	1	3	2	4	1	5	2	3
Frame 1	6	6	6	3	3	4	4	5	5	3
Frame 2		2	2	2	2	2	2	2	2	2
Frame 3			1	1	1	1	1	1	1	1
Fault / Hit	F	F	F	F	H	F	H	F	H	F
Evicted	–	–	–	6	–	3	–	4	–	5

Legend: **F** = page fault (red cell), **H** = hit (green cell); a blank frame cell means that frame is still empty. The **Evicted** row shows the page removed on a fault.

Why each Optimal eviction (look-ahead)

- Column 4 (page 3): next use – 6→never, 2→pos 5, 1→pos 7 ⇒ evict **6**.
- Column 6 (page 4): next use – 3→pos 10, 2→pos 9, 1→pos 7 ⇒ evict **3**.
- Column 8 (page 5): next use – 4→never, 2→pos 9, 1→never ⇒ evict **4**.
- Column 10 (page 3): next use – 5→never, 2→never, 1→never ⇒ evict **5**.

Faults occur at references 6, 2, 1, 3, 4, 1, 5, 3. Total page faults = 7; Hits = 3.

Because pages 1, 3 and 5 each get reused after being evicted, the count is 7 – still the minimum possible for this particular string under any algorithm.

Q8. Examine memory-reference behaviour and identify how thrashing occurs when page frames are insufficient. Suggest one control measure.

Concept: *Thrashing* | ***Key terms:*** *insufficient frames, working set*

Observed behaviour: when the frames allocated to the active processes are **fewer than their combined working set**, every process repeatedly references pages that are not resident.

How thrashing develops

1. A process references a page that is not in memory → **page fault**.
2. Because memory is full, servicing the fault **evicts another page that is also active**.
3. That evicted page is referenced again almost immediately → **another fault**.
4. The **page-fault rate soars**; disk paging dominates and CPU utilisation drops sharply even though the CPU appears busy handling faults. This self-reinforcing cycle is **thrashing**.

Control measure

Apply the working-set model: estimate each process's working set (the set of pages it is actively using) and admit or keep a process only if enough frames are available for its working set; otherwise lower the degree of multiprogramming so the remaining processes have sufficient frames.

Q9. Analyse the relationship between demand paging, swapping and thrashing in a virtual-memory system.

Concept: *VM dynamics* | ***Key terms:*** *cause-effect chain*

These three concepts form a **cause-and-effect chain** in a virtual-memory system:

1. **Demand paging** loads pages only when referenced, keeping memory use low but generating **page faults**.

2. Servicing a fault often requires **swapping** – writing a victim page out to disk and reading the needed page in.

3. If memory is **over-committed**, faults and swaps become so frequent that the system **thrashes**, and throughput collapses.

Analysis: demand paging and swapping are normal, efficient mechanisms that increase effective memory capacity; pushed beyond the physical-memory limit they degenerate into thrashing. The remedy is to keep each process's working set resident and to control the degree of multiprogramming.

Q10. Elaborate on the relationship between address translation, segmentation and paging in a virtual-memory system.

Concept: *Address translation* | ***Key terms:*** *segmentation, paging, segmented paging*

> **DEFINITION**
> **Address translation** is the process of converting a logical (virtual) address generated by the CPU into a physical address in RAM, performed by the MMU using OS-maintained tables.

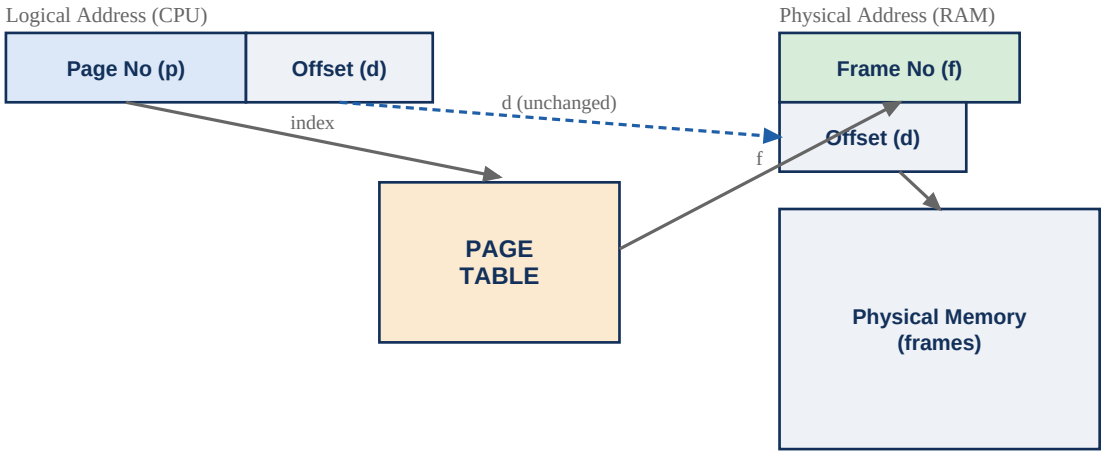


Fig: A logical address (page, offset) is translated to a physical address via the page table.

Segmentation

Divides the address space into variable-size, logically meaningful **segments** (code, data, stack). A logical address is a pair **(segment number, offset)**, translated via a **segment table** that gives a base address and a limit for each segment.

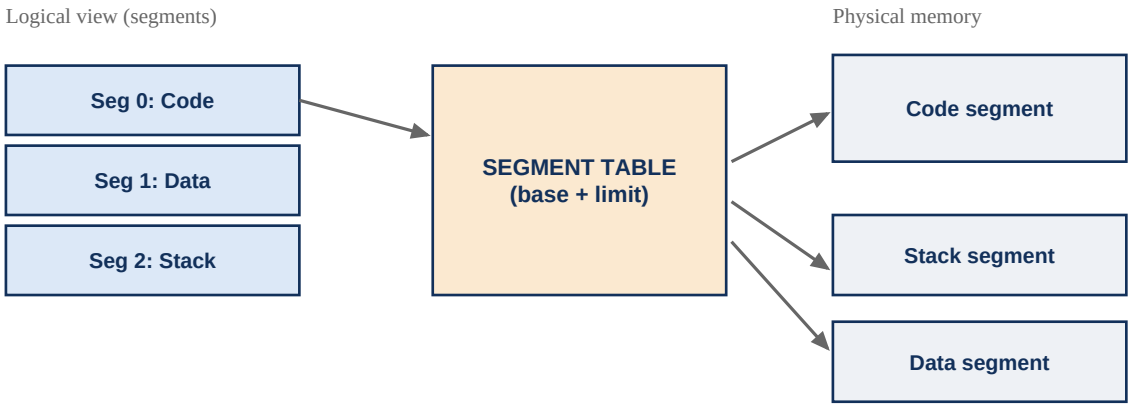


Fig: Segmentation maps variable-size logical segments to physical memory via a segment table.

Paging

Divides memory into fixed-size **pages** and **frames**. A logical address is **(page number, offset)**, translated via a **page table**. Paging removes external fragmentation.

Segmented paging (the relationship)

The two can be combined: each segment is itself paged, so a logical address becomes (segment, page, offset). This keeps the logical structure and protection of segmentation while removing external fragmentation through paging. Thus address translation is the umbrella operation, and segmentation and paging are the two schemes (often combined) that define how the mapping tables are organised.

Q11. Assess IPC and RPC as communication mechanisms for RTOS processes and remote services, with one use case each.

Concept: *IPC vs RPC use cases* | ***Key terms:*** *local coordination vs remote invocation*

IPC (shared memory, message queues, semaphores) is best for **coordinating tasks on the same node**: it is fast and low-overhead, but the programmer must manage the communication explicitly.

RPC extends the procedure-call model **across machines**: it is transparent (looks like a local call) but incurs network latency and parameter marshalling cost.

Use cases

- **IPC use case:** inside one embedded controller, a sensor-reading task passes data to a control task through a **message queue**.
- **RPC use case:** in a distributed control system, a local task requests a computation or database lookup from a **remote server** via an RPC.

Assessment: choose IPC for tight, low-latency coordination within a node, and RPC when the service genuinely resides on another machine and location transparency is valuable.

Q12. Analyse the role of segmentation and free-space management in efficient memory allocation.

Concept: *Segmentation & free-space management* | ***Key terms:*** *variable partitions, free lists*

Segmentation

Allocates memory as **variable-size segments** matching the program's logical parts (code, data, stack). This supports **sharing and protection** at segment granularity but can cause **external fragmentation** because segments differ in size.

Free-space management

Keeps track of unused memory so it can be reused efficiently, using data structures such as:

- **Bitmaps** – one bit per allocation unit marks it free or used.
- **Free lists** – a linked list of free holes, allocated using first-fit, best-fit or worst-fit.
- **Coalescing** of adjacent free holes and **compaction** to reduce fragmentation.

Role together: segmentation decides *how* memory is requested (variable logical units), while free-space management decides *where* each request is placed and how freed memory is recycled. Good free-space management keeps fragmentation and search time low, making segmentation practical.

Q13. Differentiate segmentation and free-space management as techniques for managing memory.

Concept: *Segmentation vs free-space management* | ***Key terms:*** *allocation scheme vs bookkeeping*

Aspect	Segmentation	Free-space management
Purpose	Divide the address space into logical variable-size segments	Track and reuse unused memory holes
View of memory	Logical (programmer's) view	Physical bookkeeping view
Mechanism	Segment table (base, limit)	Bitmaps / free lists + fit algorithms
Main issue	External fragmentation	Choosing a hole, coalescing, compaction
When used	Address-space organisation	During every allocate/free

In short, segmentation is a memory-*organisation* technique, while free-space management is the *bookkeeping* that supports allocation and de-allocation.

Q14. Discuss how demand paging and swapping influence the memory behaviour of an operating system.

Concept: *Effect on memory behaviour* | ***Key terms:*** *capacity vs paging overhead*

- **Demand paging** lets processes start with only a few pages resident, so **more processes fit** in memory and program startup is fast; the cost is page-fault handling on first access to each page.
- **Swapping** frees memory by moving pages or whole processes to disk, allowing a **higher degree of multiprogramming** than physical RAM alone would permit.
- Both mechanisms **increase effective memory capacity** and utilisation, but add **disk I/O overhead**.
- If overused, they raise the page-fault rate and can lead to **thrashing**.

Overall: demand paging and swapping trade a controllable amount of disk I/O for much better memory utilisation and a higher degree of multiprogramming – beneficial until memory is over-committed.

Q15. Explain how the Memory API and address spaces support memory virtualisation.

Concept: *Memory API + address space* | ***Key terms:*** *virtual allocation, isolation*

Memory virtualisation gives each process the illusion of a large, private memory. Two pieces work together to achieve this:

1. The **address space** defines each process's private range of virtual addresses (code, data, heap, stack), isolated from other processes and mapped to physical frames by the OS/MMU.
2. The **Memory API** (malloc/free, brk/sbrk, mmap) lets a program request and release regions **within that virtual address space** without knowing the physical layout.

So the API operates purely on virtual addresses, while the OS transparently backs those addresses with physical frames through demand paging. The result is isolation, protection and the illusion of abundant contiguous memory for every process.

Q16. Use the concept of a TLB to explain how repeated virtual-address translations are accelerated.

Concept: *TLB acceleration* | ***Key terms:*** *locality, hit/miss, EAT*

Without a TLB, every memory access would need an **extra memory read of the page table** before the actual data access – effectively doubling memory access time. The **TLB** caches recent page→frame mappings to avoid this.

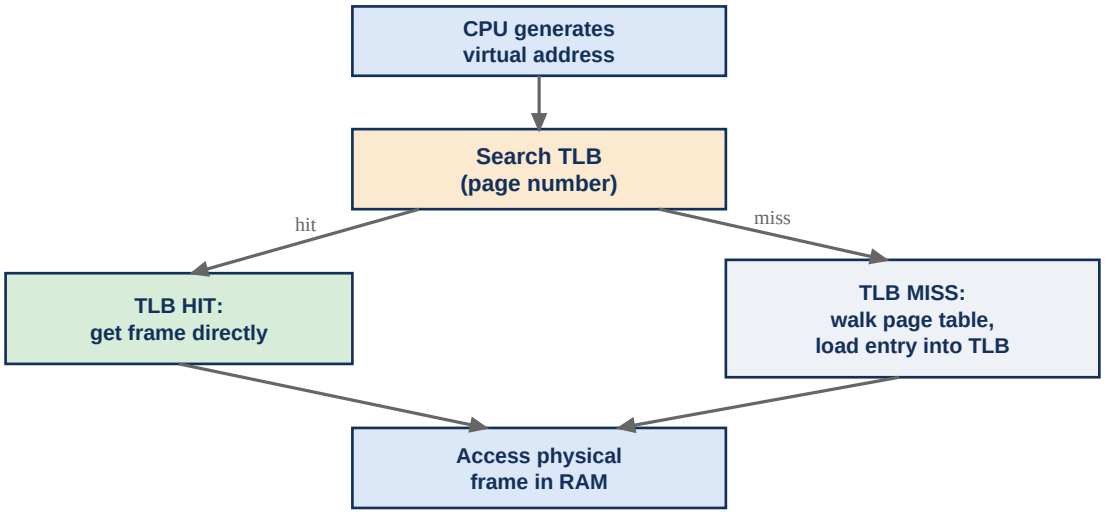


Fig: TLB hit path (fast) vs TLB miss path (page-table walk).

- On a **TLB hit**, the frame number is found in the fast TLB and the physical address is formed immediately – one memory access.
- On a **TLB miss**, the page table in memory is walked, and the new mapping is loaded into the TLB for next time.
- Because programs exhibit **locality of reference**, most translations hit the TLB, so repeated accesses to the same pages are very fast.

Effective Access Time (EAT) = $h\times(t_{TLB}+t_{mem}) + (1-h)\times(t_{TLB}+2\times t_{mem})$. A high hit ratio h keeps EAT close to a single memory access, which is how repeated translations are accelerated.

Q17. Determine how a logical address is divided into page number and offset, and indicate how the TLB accelerates the translation.

Concept: *Address splitting + TLB* | ***Key terms:*** *page number, offset, frame*

In a paged system, a logical address of **m bits** with a page size of 2^n bytes is split into two parts:

- **Page number (p)** = the high-order **(m** – **n)** bits – used to index the page table.
- **Offset (d)** = the low-order **n** bits – the position within the page, which is **unchanged** during translation.

Translation steps

1. Split the logical address into (p, d).
2. Use **p** to look up the **frame number f** in the page table (or the TLB).
3. Form the physical address as **(f, d)** – frame number concatenated with the same offset.

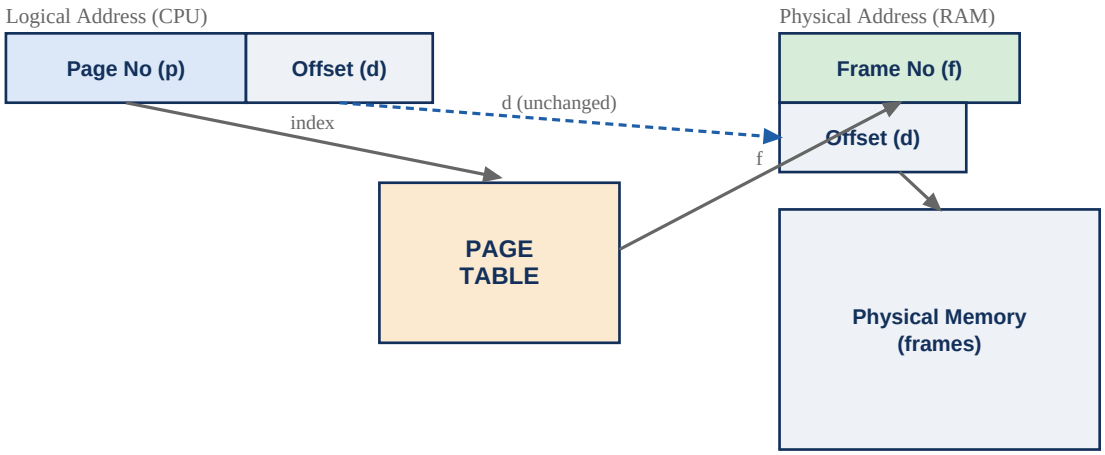


Fig: (page, offset) → (frame, offset) translation.

Example: for a 16-bit logical address with 4 KB pages (n = 12), the top 4 bits are the page number and the low 12 bits are the offset.

Role of the TLB: it caches recent (p → f) mappings, so on a TLB hit the frame number is obtained without accessing the page table in memory – accelerating the translation.

End of Detailed Study Guide — RTOS CO1 & CO2. Practise every Gantt chart and page-fault table by hand at least once. In the examination: state the definition, draw the diagram/table, show the formula and working, and clearly box the final answer to secure full marks.

Appendix A — Quick Comparison Tables

One-glance revision of the most-asked comparisons

A.1 Scheduling algorithms at a glance

Algorithm	Preemptive	Basis	Avg waiting	Starvation
FCFS	No	Arrival order	High (convoy)	No
SJF	No	Shortest burst	Lowest	Possible
SRTF	Yes	Shortest remaining time	Low	Possible
Priority	Either	Priority number	Varies	Possible (aging fixes)
Round Robin	Yes	Time quantum	Moderate	No
MLFQ	Yes	Feedback queues	Low (adaptive)	No (with aging)

A.2 Page-replacement algorithms

Algorithm	Evicts	Belady's anomaly?	Implementable?
FIFO	Oldest loaded page	Yes	Yes (simple)
LRU	Least recently used	No	Yes (costly / approximated)
Optimal	Farthest future use	No	No (needs future)
LFU	Least frequently used	Possible	Yes

A.3 Contiguous allocation strategies

Strategy	Chooses	Pros	Cons
First-Fit	First hole that fits	Fast, simple	Leaves fragments near start
Best-Fit	Smallest hole that fits	Least leftover	Many tiny unusable holes; slow
Worst-Fit	Largest hole	Large usable leftover	Often worst utilisation

A.4 Paging vs Segmentation

Aspect	Paging	Segmentation
Block size	Fixed (page = frame)	Variable (logical segment)
Divided by	Hardware	Logical/user structure
Fragmentation	Internal	External
Address	(page no, offset)	(segment no, offset)
Table	Page table	Segment table

A.5 GPOS vs RTOS

Aspect	GPOS	RTOS
Goal	Throughput, fairness	Meet deadlines
Response time	Variable	Bounded, guaranteed
Scheduling	Fairness-based	Priority / deadline-based

Examples	Windows, Linux	FreeRTOS, VxWorks, QNX

Appendix B — Formula & Fact Sheet

Everything you must recall in the exam

Scheduling

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time
- Response Time = First-CPU-allocation Time – Arrival Time
- Average WT = (Σ WT) / n ; Average TAT = (Σ TAT) / n
- CPU utilisation = (busy time / total time) × 100%
- Throughput = number of processes completed / total time

Paging & virtual memory

- Hit ratio = hits / total references
- Page-fault ratio = page faults / total references
- Number of pages = process size / page size
- Physical address = (frame number × frame size) + offset
- EAT (no TLB) = 2 × memory access time (one for page table, one for data)
- EAT (with TLB) = $h(t_{TLB} + t_{mem}) + (1–h)(t_{TLB} + 2t_{mem})$

Real-time (RMS)

- CPU utilisation $U = \Sigma (C_i / T_i)$
- RMS schedulability bound (n tasks): $U \leq n(2^{1/n} – 1)$
- For large n the bound $\rightarrow \ln 2 \approx 0.693$
- EDF schedulability bound: $U \leq 1$ (up to 100% utilisation)

Handy facts to quote

- SJF gives the **minimum average waiting time** among non-preemptive algorithms.
- **Belady's anomaly** affects FIFO but not stack algorithms (LRU, Optimal).
- **Optimal** gives the theoretical minimum page faults but needs future knowledge.
- Paging removes **external** fragmentation but causes small **internal** fragmentation.
- RMS is the **optimal fixed-priority** real-time scheduling algorithm.

Appendix C — Glossary of Key Terms

Short, exam-ready definitions

Term	Meaning
Operating System	System software that manages hardware and provides services to programs; acts as resource manager and extended machine.
Kernel	The privileged core of the OS that controls processes, memory and devices.
Process	A program in execution, represented by a Process Control Block (PCB).
Context switch	Saving one process's state and loading another's so the CPU can switch between them.
Scheduler	OS component that selects the next process to run on the CPU.
Dispatcher	OS component that performs the context switch to the selected process.
Burst time	The amount of CPU time a process needs in one go.
Turnaround time	Total time from a process's arrival to its completion.
Waiting time	Total time a process spends in the ready queue.
Response time	Time from arrival to the first CPU allocation.
Pre-emption	Forcibly taking the CPU away from a running process.
Starvation	Indefinite waiting of a process for the CPU; fixed by aging.
Aging	Gradually increasing the priority of long-waiting processes.
Paging	Non-contiguous memory scheme using fixed-size pages and frames.
Frame	A fixed-size block of physical memory.
Page	A fixed-size block of logical memory.
Page table	Per-process table mapping page numbers to frame numbers.
Page fault	A reference to a page not currently in main memory.
TLB	Fast cache of recent page-to-frame mappings, speeding translation.
Demand paging	Loading pages into memory only when first referenced.
Swapping	Moving processes/pages between memory and disk.
Thrashing	Excessive paging where the CPU does little useful work.
Working set	The set of pages a process is actively using.
Segmentation	Memory scheme dividing the address space into variable-size logical segments.
External fragmentation	Free memory split into holes too small to use.
Internal fragmentation	Wasted space inside an allocated block.
Address space	The range of logical addresses a process may use.
RTOS	OS that guarantees task completion within strict deadlines.
Hard real-time	A missed deadline is a total, often catastrophic, failure.
Soft real-time	Deadlines should be met; occasional misses only degrade quality.
RMS	Rate-Monotonic Scheduling: shorter period gets higher fixed priority.
EDF	Earliest-Deadline-First: the nearest deadline runs next (dynamic priority).
Priority inversion	A high-priority task blocked by a lower-priority one holding a resource.
IPC	Inter-Process Communication between processes on the same machine.

RPC	Remote Procedure Call: invoking a procedure on another machine transparently.

Appendix D — Likely Exam & Viva Questions

Practise these to be exam-ready

Frequently asked long-answer questions

1. Explain the different CPU-scheduling algorithms with examples and compare them.
2. Solve a given set of processes using FCFS / SJF / Priority / Round Robin and compute average waiting and turnaround times.
3. Explain paging with a neat diagram and show how logical addresses are translated to physical addresses.
4. Compare FIFO, LRU and Optimal page replacement for a given reference string.
5. Explain demand paging, swapping and thrashing, and how thrashing is controlled.
6. Explain segmentation and compare it with paging.
7. Explain Rate-Monotonic Scheduling and perform the schedulability test for a task set.
8. Differentiate a general-purpose OS from a real-time OS.

Common viva / short questions

1. What is the difference between turnaround time and waiting time?
2. What is Belady's anomaly and which algorithm suffers from it?
3. Why is the Optimal algorithm not implementable?
4. What is the purpose of the TLB?
5. What causes thrashing and how is it avoided?
6. What is the difference between internal and external fragmentation?
7. Why does SJF give minimum average waiting time?
8. What is priority inversion and how is it solved?
9. What is the difference between hard and soft real-time systems?
10. What is a context switch and when does it happen?

> **EXAM TIP**
> If you can confidently answer every question in this appendix, you are well prepared. Focus your final revision on the worked numericals and the comparison tables – they appear in almost every paper.

Appendix E — Extra Solved Practice Problems

Additional worked examples for exam practice

These problems are **not** from the question bank; they are extra practice covering the same methods so you can rehearse before the exam. Cover the solution, solve it yourself, then check.

E.1 Round Robin (time quantum = 2)

Processes (all arrive at t = 0): P1(BT 5), P2(BT 4), P3(BT 3).

Gantt chart – each process gets 2 units in turn until its burst is exhausted:

P1	P2	P3	P1	P2	P3	P1

Timeline: 0 → 2 → 4 → 6 → 8 → 10 → 11 → 12

Completion: P3 = 11, P2 = 10, P1 = 12.

Process	AT	BT	Completion	Turnaround	Waiting
P1	0	5	12	12	7
P2	0	4	10	10	6
P3	0	3	11	11	8
Average				**11.0**	**7.0**

Average Waiting Time = 21/3 = 7.0 units; Average Turnaround Time = 33/3 = 11.0 units.

E.2 Preemptive SJF – Shortest Remaining Time First (SRTF)

Processes: P1(AT 0, BT 8), P2(AT 1, BT 4), P3(AT 2, BT 2).

At each unit the process with the **shortest remaining time** runs; a shorter arrival can pre-empt.

P1	P2	P3	P2	P1

Timeline: 0 → 1 → 2 → 4 → 7 → 14

Reasoning: P1 starts; at t=1 P2 (4) < P1 remaining (7), so P2 runs; at t=2 P3 (2) is shortest, runs to t=4; then P2 finishes (t=7); finally P1 (7 left) runs to t=14.

Process	AT	BT	Completion	Turnaround	Waiting
P1	0	8	14	14	6
P2	1	4	7	6	2
P3	2	2	4	2	0
Average				**7.33**	**2.67**

Average Waiting Time = 8/3 ≈ 2.67 units; Average Turnaround Time = 22/3 ≈ 7.33 units.

E.3 Preemptive Priority (lower number = higher priority)

Processes: P1(AT 0, BT 4, Pri 2), P2(AT 1, BT 3, Pri 1), P3(AT 2, BT 2, Pri 3).

A higher-priority arrival pre-empts the running process.

P1	P2	P1	P3

Timeline: 0 → 1 → 4 → 7 → 9

Reasoning: P1 runs; at t=1 P2 (Pri 1) pre-empts P1 and runs to completion (t=4); then P1 resumes its remaining 3 units (t=4–7); finally P3 runs (t=7–9).

Process	AT	BT	Pri	Completion	Turnaround	Waiting
P1	0	4	2	7	7	3
P2	1	3	1	4	3	0
P3	2	2	3	9	7	5
Average					**5.67**	**2.67**

Average Waiting Time = 8/3 ≈ 2.67 units; Average Turnaround Time = 17/3 ≈ 5.67 units.

E.4 Comparing FIFO, LRU and Optimal on one string

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, with **3 frames**. Work all three and compare.

(a) FIFO (3 frames) – evict the oldest page:

Ref	F1	F2	F3	Fault?
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
4	4	2	3	Fault
1	4	1	3	Fault
2	4	1	2	Fault
5	5	1	2	Fault
1	5	1	2	Hit
2	5	1	2	Hit
3	3	1	2	Fault
4	3	4	2	Fault
5	3	4	5	Fault

FIFO page faults = **9**.

(b) LRU (3 frames) – evict the least recently used page:

Ref	F1	F2	F3	Fault?
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
4	4	2	3	Fault
1	4	1	3	Fault

2	4	1	2	Fault
5	5	1	2	Fault
1	5	1	2	Hit
2	5	1	2	Hit
3	3	1	2	Fault
4	3	4	2	Fault
5	3	4	5	Fault

LRU page faults = **10**.

(c) Optimal (3 frames) – evict the page used farthest in the future:

Ref	F1	F2	F3	Fault?
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
4	1	2	4	Fault
1	1	2	4	Hit
2	1	2	4	Hit
5	1	2	5	Fault
1	1	2	5	Hit
2	1	2	5	Hit
3	3	2	5	Fault
4	4	2	5	Fault
5	4	2	5	Hit

Optimal page faults = **7**.

Algorithm	Page faults (3 frames)
FIFO	9
LRU	10
Optimal	7

Optimal gives the fewest faults, as expected. Note that a 'smarter' policy is not automatically better on *every* string – here LRU happens to do slightly worse than FIFO. This same string is the classic demonstration of Belady's anomaly: with FIFO and 4 frames it produces 10 faults – *more* than with 3 frames – even though memory increased.

E.5 Paging address-translation numerical

Given: logical address space = 16 pages, page size = 1 KB (1024 bytes), physical memory = 32 frames. Find the number of bits for the page number, offset, and physical address.

1. **Offset bits** = $\log_2$(page size) = $\log_2$(1024) = **10 bits**.
2. **Page-number bits** = $\log_2$(number of pages) = $\log_2$(16) = **4 bits**.
3. So the **logical address** = 4 + 10 = **14 bits**.
4. **Frame-number bits** = $\log_2$(32) = **5 bits**.
5. So the **physical address** = 5 + 10 = **15 bits**.

Translation example: if page 5 maps to frame 20, then logical address (page 5, offset 100) → physical address = 20 × 1024 + 100 = 20480 + 100 = 20580.

E.6 Effective Access Time (EAT) with a TLB

Given: TLB access time = 20 ns, memory access time = 100 ns, TLB hit ratio = 80%.

On a hit: time = TLB + memory = 20 + 100 = 120 ns.
On a miss: time = TLB + page-table access + memory = 20 + 100 + 100 = 220 ns.

EAT = 0.80 × 120 + 0.20 × 220 = 96 + 44 = 140 ns.

Without a TLB every access would take 200 ns (two memory accesses), so the TLB with an 80% hit ratio reduces the effective time to 140 ns.

E.7 Rate-Monotonic Scheduling – schedulability

Given tasks: T1(C=1, T=4), T2(C=2, T=6), T3(C=1, T=8). Assign RMS priorities and test schedulability.

1. By RMS, shortest period = highest priority: **T1 > T2 > T3**.
2. Utilisation U = 1/4 + 2/6 + 1/8 = 0.25 + 0.333 + 0.125 = **0.708**.
3. Bound for n=3: $3(2^{1/3} - 1) \approx$ **0.779**.
4. Since 0.708 ≤ 0.779, the task set is **schedulable under RMS**.

Conclusion: all three tasks will meet their deadlines; T1 pre-empts T2 and T3 whenever it is ready, and T3 runs only in the gaps left by T1 and T2.

> **EXAM TIP**
> For RMS problems always: (1) order tasks by period, (2) compute U = ΣC/T, (3) compare with n(2^(1/n)–1). State clearly whether U is within the bound.

E.8 First-Fit vs Best-Fit vs Worst-Fit on one problem

Free blocks (in order): 100, 500, 200, 300, 600 KB. Requests (in order): 212, 417, 112, 426 KB. Apply all three strategies.

First-Fit (first block that fits):

Request	Allocated to	Remaining
212 KB	500 → 288	288 KB
417 KB	500? used; 600 → 183	183 KB
112 KB	288 (leftover) → 176	176 KB
426 KB	no single hole ≥ 426 left	Waits / fails

Best-Fit (smallest hole that fits): 212→300, 417→500, 112→200, 426→600 – **all allocated**.

Worst-Fit (largest hole): 212→600, 417→500, 112→388(leftover of 600), 426→ largest left = 300 → **fails**.

Observation: for this data Best-Fit satisfies all four requests, while First-Fit and Worst-Fit fail on the 426 KB request. This shows how the placement strategy directly affects whether a request can be met – a frequent exam discussion point.

Appendix F — Common Mistakes to Avoid

Lose no easy marks in the exam

Scheduling problems

- Forgetting that **Turnaround = Completion** – **Arrival** (students often forget to subtract arrival time when AT ≠ 0).
- Mixing up **waiting time** and **response time** – response time is only up to the *first* CPU allocation.
- In pre-emptive algorithms, not re-checking the ready queue at **every** new arrival.
- Not drawing the Gantt chart – the working carries most of the marks.

Page-replacement problems

- Confusing **FIFO** (evict oldest *loaded*) with **LRU** (evict least recently *used*).
- In LRU, updating recency on a **hit** is essential – a hit still counts as a use.
- In Optimal, looking at **future** references (not past) to choose the victim.
- Forgetting to count the initial fills as page faults (empty frames still cause faults).

Memory-allocation problems

- Not updating the **remaining block size** after each allocation.
- Confusing **internal** and **external** fragmentation.
- For Worst-Fit, forgetting that a leftover piece becomes a new (smaller) hole.

Real-time / RTOS

- Reversing the RMS rule – it is **shorter period = higher priority**.
- Forgetting the schedulability bound n(2^(1/n)–1) and just comparing U with 1 (that is EDF).
- Confusing **hard** and **soft** real-time in definitions.

> **EXAM TIP**
> Neatness matters: label every Gantt chart and frame table, underline final answers, and always write the formula before substituting numbers. Presentation earns method marks even if a small arithmetic slip occurs.